whereas you'll normally update `Startup` whenever you add additional features. For example, if you add a new NuGet dependency to your project, you'll normally need to update `Startup` to make use of it.

The `Program` class is where a lot of app configuration takes place, but this is mostly hidden inside the `Host.CreateDefaultBuilder` method. `CreateDefaultBuilder` is a static helper method that simplifies the bootstrapping of your app by creating an `IHostBuilder` with some common configuration. This is similar to the way you've used `WebApplication.CreateDefaultBuilder()` throughout the book.

NOTE You can create custom `HostBuilder` instances if you want to customize the default setup and create a completely custom `IHost` instance, as you'll see in section 30.4. This is different from `WebApplicationBuilder`, which always uses the same defaults.

The other helper method used by default is `ConfigureWebHostDefaults`. This uses a `WebHostBuilder` object to configure Kestrel to listen for HTTP requests.

Creating services with the generic host

It might seem strange that you must call `ConfigureWebHostDefaults` as well as `CreateDefaultBuilder`. Couldn't we have one method? Isn't handling HTTP requests the whole point of ASP.NET Core?

Well, yes and no! ASP.NET Core 3.0 introduced the concept of a generic host. This allows you to use much of the same framework as ASP.NET Core applications to write non-HTTP